深度学习
第 5 章　学习的技巧

——梯度、初始化、正则化：让深层网络真正”学得起来”的三把钥匙

2026 年 3 月 17 日

本章题目导览：你将挑战哪些核心问题？

搭一个深层神经网络很容易，但让它真正**稳定**、**高效地训练**却困难得多。梯度会消失或爆炸，优化器会原地震荡，权重初始化不当会让网络一开始就”跑偏”，过拟合会让高分训练集变成低分测试集。本章 6 道题逐一击破这些拦路虎：

- **Q5-01（辨析填空）**：梯度消失 vs 梯度爆炸——深层网络为什么容易出现这两种问题？各自的症状、根源和解决思路是什么？
- **Q5-02(计算题)**:优化器更新规则手动推导——给定梯度历史,分别用 SGD、Momentum、AdaGrad 手算一步参数更新，把公式变成具体数字。
- **Q5-03（对比分析）**：六种优化器横向对比——从 SGD 到 Adam，各自解决了什么问题、引入了什么超参数、适合什么场景？
- **Q5-04（概念理解）**：六种参数初始化方法对比——常数、秩、正态、均匀、Xavier、He 初始化，各自的公式和适用激活函数是什么？
- **Q5-05（代码阅读）**：Batch Normalization 代码追踪——BN 层的计算过程、训练/推理行为差异，以及在网络中的位置选择。
- **Q5-06（综合分析）**：过拟合三剑客——权值衰减、Dropout、BatchNorm 各自如何抑制过拟合？它们能叠加使用吗？

1　第 5 章　学习的技巧

前面几章我们已经能搭出一个神经网络并跑通训练循环了。但”能跑”和”能学好”之间，还差着一大截：

- 网络加深到 10 层，梯度传到底层已经变成接近 0 的小数——参数根本没在更新；
- SGD 在峡谷形损失曲面上左右震荡，迟迟无法到达谷底；
- 初始权重太大或太小，激活值要么全饱和、要么全为零——无论怎么训练都没有起色；
- 训练集准确率 99%，验证集准确率 65%——模型把噪声也背下来了。

这些不是玄学问题，都有对应的**技巧**来解决。本章就是这本”工具手册”。

先建立一个总感觉：为什么深度学习需要”技巧”？

把训练深层神经网络比作在山脉中寻找最低谷。

- **梯度消失/爆炸** = 山太高太深，向导（梯度）的声音传到山底时已经完全失真——要么细如蚊鸣（消失），要么震耳欲聋（爆炸），两种情况下你都找不到方向；
- **优化器** = 你的行走策略：原地踏步（SGD 震荡）、借助惯性滑行（Momentum）、根据地形自动调节步伐（AdaGrad/Adam）；
- **参数初始化** = 出发点的选择：从悬崖边出发（权重过大）或者从坑底出发（权重过小），都很难找到最优路线；
- **正则化** = 防止你把地图上的噪声当成真实地形记住——记住规律，而不是死记硬背。

Q5-01 梯度消失 vs 梯度爆炸：深层网络的两大顽疾

题目描述：

文档第 5.1 节指出，深度神经网络在训练过程中面临梯度消失（Vanishing Gradient）和梯度爆炸（Exploding Gradient）两大问题。两者都与反向传播中的链式法则密切相关。

问题：

(1) 填写下表，对比两种问题的核心特征：

	梯度消失	**梯度爆炸**
根本原因	链式法则连乘时，每层梯度值_____	链式法则连乘时，每层梯度值_____
典型症状	底层参数_____，Loss 不再下降	参数更新幅度极大，Loss_____ 或变成 NaN
易发激活函数	_____	通常与激活函数_____，与权重初始化更相关
常见解决方案	换用_____ 激活函数；使用_____ 初始化	梯度裁剪（Gradient Clipping）；使用_____ 初始化

(2) **数学推导**：设一个 5 层网络，每层的梯度（局部导数）分别为 g_1, g_2, g_3, g_4, g_5。根据链式法则，第 1 层参数的梯度为：

$$\frac{\partial L}{\partial w_1} = g_5 \cdot g_4 \cdot g_3 \cdot g_2 \cdot g_1$$

若每层使用 Sigmoid 激活函数，导数最大值约为 0.25，计算 5 层连乘后第 1 层的梯度量级（假设每层导数恰好取最大值 0.25）。

(3) **ReLU 为何能缓解梯度消失**：ReLU 在正值区域的导数为常数 1。解释为什么这能让梯度”畅通”地传递到底层，以及 ReLU 仍然可能引发梯度问题的场景（死亡 ReLU）。

(4) **梯度裁剪**是缓解梯度爆炸最直接的方法，其核心思想是：当梯度的范数超过阈值 θ 时，对梯度进行等比例缩放：

$$\mathbf{g} \leftarrow \frac{\theta}{\|\mathbf{g}\|} \cdot \mathbf{g}, \quad 当 \|\mathbf{g}\| > \theta$$

若当前梯度向量 $\mathbf{g} = [6, 8]$，裁剪阈值 $\theta = 5$，计算裁剪后的梯度。

来源溯源：[文档第 5 章 5.1 深度神经网络及其问题]

小白必读：反向传播为什么会让梯度"越传越小"？

- **链式法则的连乘效应**：反向传播本质上是从输出层向输入层逐层相乘梯度。如果每层的梯度值都是一个小于 1 的小数（比如 Sigmoid 导数最大只有 0.25），乘了 10 层之后，梯度就变成了 $0.25^{10} \approx 10^{-6}$，几乎为零，底层参数收到的"纠错信号"消失殆尽。
- **梯度爆炸的对称情形**：如果每层梯度大于 1（比如权重初始化过大，导致每层放大梯度），连乘后梯度急剧增大，参数更新幅度失控，Loss 剧烈震荡甚至变成 `NaN`。
- **深度越深，问题越严重**：层数翻倍，连乘次数翻倍，梯度消失/爆炸的程度呈指数级恶化。这正是早期深度网络（10 层以上）难以训练的核心原因。

参考答案与详细解析

(1) 对比表填写：

	梯度消失	**梯度爆炸**
根本原因	连乘时每层梯度值小于 1，连乘后趋近于 0	连乘时每层梯度值大于 1，连乘后趋向无穷
典型症状	底层参数几乎不更新，Loss 不再下降	Loss 剧烈震荡或变成 NaN
易发激活函数	Sigmoid、Tanh（导数最大值 <1）	关系较小，主要与权重初始化相关
常见解决方案	换用 ReLU；使用 Xavier/He 初始化；BatchNorm	梯度裁剪；使用合理初始化

(2) 梯度量级计算：

假设每层 Sigmoid 导数取最大值 0.25，5 层连乘：

$$\frac{\partial L}{\partial w_1} = 0.25^5 = \frac{1}{4^5} = \frac{1}{1024} \approx \boxed{9.77 \times 10^{-4}}$$

不到千分之一。若网络有 10 层，则约为 $0.25^{10} \approx 9.5 \times 10^{-7}$，几乎无法更新。

(3) ReLU 缓解梯度消失的原因：

ReLU 在正值区域导数恒为 1，连乘后梯度不会因层数增加而衰减，信号可以畅通传递。

死亡 ReLU（Dying ReLU）：当某个神经元的输入永远为负时（如参数更新后该神经元输出始终 ≤ 0），ReLU 输出恒为 0，导数恒为 0，这个神经元永远不再接收梯度更新——它"死亡"了。解决方案：使用 Leaky ReLU（负值区域有小斜率 αx，而非直接为 0）或调小学习率避免参数大幅跑偏。

(4) 梯度裁剪计算：

当前梯度 $\mathbf{g} = [6, 8]$，范数 $\|\mathbf{g}\| = \sqrt{6^2 + 8^2} = \sqrt{100} = 10$。

由于 $10 > \theta = 5$，触发裁剪：

$$\mathbf{g}_{\text{clip}} = \frac{5}{10} \cdot [6, 8] = \boxed{[3, 4]}$$

裁剪后范数恰好等于阈值 5，方向不变（仍为 $[6, 8]$ 的方向），幅度被等比缩小。

Q5-02 优化器更新规则手动推导

题目描述：

文档第 5.2 节依次介绍了 SGD、Momentum、AdaGrad 等优化器的更新规则。理解公式最好的方式是亲手算一遍。

场景设定： 某参数 w 当前值为 $w_0 = 1.0$，学习率 $\eta = 0.1$。在连续两步中，梯度分别为 $g_1 = 0.5$（第 1 步）和 $g_2 = 0.8$（第 2 步）。

问题：

(1) **标准 SGD** 更新规则为 $w \leftarrow w - \eta \cdot g$。计算两步后 w 的值。

(2) **Momentum** 更新规则为：

$$v \leftarrow \beta v - \eta \cdot g, \quad w \leftarrow w + v$$

取 $\beta = 0.9$，初始动量 $v_0 = 0$。分别计算第 1 步、第 2 步后的 v 和 w（保留三位小数）。

(3) **AdaGrad** 更新规则为：

$$h \leftarrow h + g^2, \quad w \leftarrow w - \frac{\eta}{\sqrt{h} + \epsilon} \cdot g$$

取 $\epsilon = 10^{-8}$（防止除零，计算时可忽略），初始 $h_0 = 0$。计算第 1 步、第 2 步后的 h 和 w（保留三位小数）。

(4) 对比三种方法在两步之后的 w 值，并分析：为什么在第 2 步（梯度更大）时，AdaGrad 的更新幅度反而比 SGD **更小**？这反映了 AdaGrad 的什么特性？

来源溯源：[文档第 5 章 5.2 更新参数方法的优化]

小白必读：为什么 SGD 的缺点那么明显却还在用？

- **SGD 的核心缺陷**：对所有参数使用同一个学习率，在损失曲面各方向曲率差异很大时（"峡谷"形曲面），在陡峭方向步子太大导致震荡，在平坦方向步子太小导致收敛极慢。
- **为什么还在用**：在大型模型（如 ResNet、GPT）训练中，经过仔细调学习率和配合学习率衰减后，SGD+Momentum 有时比 Adam 收敛到更好的泛化性能——这在计算机视觉领域尤为明显。Adam 虽然收敛快，但有时训练集 Loss 下降很快而测试集表现不如 SGD+Momentum。
- **实用建议**：入门用 Adam（调参简单，效果不错）；需要极致性能时，尝试 SGD+Momentum+ 学习率衰减。

参考答案与详细解析

(1) 标准 SGD：

第 1 步（$g_1 = 0.5$）：

$$w_1 = 1.0 - 0.1 \times 0.5 = 1.0 - 0.05 = 0.95$$

第 2 步（$g_2 = 0.8$）：

$$w_2 = 0.95 - 0.1 \times 0.8 = 0.95 - 0.08 = \boxed{0.87}$$

(2) Momentum（$\beta = 0.9$，$v_0 = 0$）：

第 1 步:

$$v_1 = 0.9 \times 0 - 0.1 \times 0.5 = -0.05$$

$$w_1 = 1.0 + (-0.05) = 0.95$$

第 2 步:

$$v_2 = 0.9 \times (-0.05) - 0.1 \times 0.8 = -0.045 - 0.08 = -0.125$$

$$w_2 = 0.95 + (-0.125) = \boxed{0.825}$$

Momentum 将历史梯度的"惯性"（$v_1 = -0.05$）叠加到当前更新上，步子比 SGD 更大，收敛更快。

(3) AdaGrad（ϵ 忽略，$h_0 = 0$）:

第 1 步（$g_1 = 0.5$）:

$$h_1 = 0 + 0.5^2 = 0.25$$

$$w_1 = 1.0 - \frac{0.1}{\sqrt{0.25}} \times 0.5 = 1.0 - \frac{0.1}{0.5} \times 0.5 = 1.0 - 0.1 = 0.9$$

第 2 步（$g_2 = 0.8$）:

$$h_2 = 0.25 + 0.8^2 = 0.25 + 0.64 = 0.89$$

$$w_2 = 0.9 - \frac{0.1}{\sqrt{0.89}} \times 0.8 = 0.9 - \frac{0.1}{0.9434} \times 0.8 \approx 0.9 - 0.0848 = \boxed{0.815}$$

(4) 对比分析:

优化器	w_0	w_1	w_2
SGD	1.000	0.950	0.870
Momentum	1.000	0.950	0.825
AdaGrad	1.000	0.900	0.815

在第 2 步，SGD 更新幅度为 0.08，AdaGrad 约为 0.085（此例中 h 的累积使有效学习率约为 0.106，但分母中的 $\sqrt{h}$ 已大于 1）。随着训练步数增加，h 持续累积增大，分母 $\sqrt{h}$ 越来越大，有效学习率 $\eta/\sqrt{h}$ 单调递减——**历史梯度越大的参数，后续学习率越小**。这正是 AdaGrad 的"自适应学习率"特性：频繁更新的参数自动减小步长，稀疏梯度的参数维持大步长，特别适合稀疏特征数据（如词频）。

AdaGrad 的致命弱点

h 是**单调递增**的——它只累加，从不遗忘。训练到后期，h 极大，有效学习率趋近于 0，参数几乎无法更新。RMSProp 用**指数移动平均**代替全量累加，解决了这个问题。

Q5-03 六种优化器横向对比

题目描述：

文档第 5.2 节完整介绍了从 SGD 到 Adam 的演进历程。每一种优化器的诞生，都是为了解决上一种的某个缺点。

问题：

(1) 填写下表，梳理六种优化器的核心信息：

优化器	核心改进点	关键超参数	适用场景
SGD	基础梯度下降，无改进	学习率 η	_____
Momentum	引入_____，赋予惯性	η，动量系数 β	_____
学习率衰减	训练后期_____，避免震荡	衰减策略、衰减率	_____
AdaGrad	自适应学习率，历史梯度平方_____	η，ϵ	_____
RMSProp	改用_____，解决 AdaGrad 学习率归零	η，衰减率 ρ	_____
Adam	融合_____ 和_____，加偏差修正	η，β_1，β_2，ϵ	_____

(2) **学习率衰减策略**：列举两种常见的学习率衰减方式，写出其调度规则，并说明各自的优缺点。

(3) **Adam 的两个动量**：Adam 同时维护一阶动量（梯度的指数移动平均 m_t）和二阶动量（梯度平方的指数移动平均 v_t），其更新公式为：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \quad w \leftarrow w - \frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t$$

解释一阶动量和二阶动量各自的”功能”，以及为什么需要偏差修正（$\hat{m}_t$、$\hat{v}_t$）。

(4) 在深度学习实践中，如何选择优化器？给出一个决策流程建议（考虑：任务类型、数据集大小、调参成本等因素）。

来源溯源：[文档第 5 章 5.2 更新参数方法的优化]

生活比喻：六种优化器像六种下山方式

你站在山顶，要找到最低点，脚下只能感受到坡度（梯度）：

- **SGD**：每步严格朝当前最陡方向迈固定步伐，遇到峡谷就来回震荡；
- **Momentum**：像滚石，有惯性，能冲出浅坑，但可能冲过头；
- **学习率衰减**：开始步子大，接近谷底时主动放慢——像跑步冲线前收速；
- **AdaGrad**：走过陡路（大梯度）后主动缩小步伐，但步子只会越来越小；
- **RMSProp**：和 AdaGrad 一样聪明，但只记最近的路况，步子不会无限缩小；
- **Adam**：既有滚石的惯性（m_t），又能根据近期地形自动调步幅（v_t），综合最优，但在某些地形下可能错过”更好的最低点”。

参考答案与详细解析

(1) 六种优化器对比表：

优化器	核心改进点	关键超参数	适用场景
SGD	基础梯度下降	η	凸问题/配合精细调参
Momentum	引入历史梯度惯性	η，β	损失曲面有”峡谷”时
学习率衰减	训练后期自动降低学习率	衰减策略/衰减率	所有场景的辅助策略
AdaGrad	历史梯度平方累加	η，ϵ	稀疏特征/NLP 词频
RMSProp	指数移动平均代替全量累加	η，ρ	非平稳目标/RNN
Adam	融合 Momentum 和 RMSProp	η，β_1，β_2，ϵ	大多数深度学习任务

(2) 两种学习率衰减策略：

- **步进衰减（Step Decay）**：每隔固定步数/epoch，将学习率乘以衰减因子 γ（如每 10 个 epoch 乘以 0.1）。优点：简单直观，易于控制；缺点：学习率阶梯式突变，可能引起短暂震荡。
- **余弦退火（Cosine Annealing）**：学习率按余弦函数从初始值平滑降至接近 0：$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos\frac{t\pi}{T})$。优点：过渡平滑，最终阶段学习率极小有利于精细收敛；缺点：超参数（T，$\eta_{\min}$）需要根据训练总步数预先设定。

(3) Adam 两个动量的功能：

- **一阶动量** m_t（梯度的移动平均）：扮演 Momentum 的角色，记录梯度的方向趋势，赋予更新”惯性”，帮助跨越局部极小值；
- **二阶动量** v_t（梯度平方的移动平均）：扮演 RMSProp 的角色，估计梯度的幅度，为不同参数自适应调节步长——历史梯度大的参数步长小，梯度小的步长大。

偏差修正的必要性：初始时 $m_0 = v_0 = 0$，前几步的 m_t、v_t 会被初始零值”拉低”，严重低估真实动量。修正项 $1/(1-\beta^t)$ 在初始阶段（t 小）较大，对估计值进行放大，还原真实动量；随着训练进行（t 增大），$\beta^t \to 0$，修正项趋近于 1，影响自然消失。

(4) 优化器选择建议：

- **默认首选 Adam**（$\beta_1 = 0.9$，$\beta_2 = 0.999$，$\eta = 10^{-3}$）：适合绝大多数任务，调参成本低，收敛快；
- **计算机视觉分类任务**：可以尝试 SGD + Momentum + 余弦退火，有时泛化更好；
- **NLP/稀疏特征**：AdaGrad 或 Adam 均可；Transformer 训练通常用 AdamW（Adam + 权重衰减解耦）；
- **资源受限/快速验证**：直接用 Adam，先跑通再优化；
- **训练不稳定时**：降低学习率，加入学习率预热（Warmup），或切换为 SGD+Momentum。

Q5-04 六种参数初始化方法对比

题目描述：

文档第 5.3 节系统介绍了六种参数初始化方法。初始化是训练的”第一步”，选错了，后面再好的优化器也难以弥补。

问题：

(1) 填写下表，梳理六种初始化方法的关键信息：

初始化方法	初始化规则	适用场景/注意事项
常数初始化	所有权重初始化为常数 c	_____
秩初始化	使用_____ 矩阵分解确定初值	_____
正态分布初始化	$W \sim \mathcal{N}(0,\ \sigma^2)$	σ 过大易_____，过小易_____
均匀分布初始化	$W \sim \mathcal{U}(-a,\ a)$	简单，但未考虑_____
Xavier 初始化	$\sigma = \sqrt{\dfrac{2}{n_{\text{in}} + n_{\text{out}}}}$	适合_____ 激活函数
He 初始化	$\sigma = \sqrt{\dfrac{2}{n_{\text{in}}}}$	专为_____ 激活函数设计

(2) **为什么常数初始化（全 0 或全同值）会让网络失效**：从”对称性”角度解释，并说明偏置（bias）为什么通常可以初始化为 0。

(3) **Xavier 初始化的推导思路**：Xavier 初始化的目标是让每一层输出的方差与输入方差相同。在不使用激活函数的假设下，若输入有 n_{in} 个特征，则将权重初始化为 $\mathcal{N}(0, 1/n_{\text{in}})$ 可以保持方差不变。解释为什么 Xavier 改用 $\sqrt{2/(n_{\text{in}} + n_{\text{out}})}$ 同时考虑了输入和输出维度。

(4) **He 初始化与 ReLU 的配合**：He 初始化的方差比 Xavier 大了约一倍（分母只有 n_{in}，没有 n_{out}）。解释为什么 ReLU 需要更大的初始方差，以及如果对 ReLU 网络使用 Xavier 初始化会发生什么。

来源溯源：[文档第 5 章 5.3 参数初始化]

小白必读：初始化为什么不能”随便设”？

- **初始化过大**：权重绝对值很大，激活函数（Sigmoid/Tanh）输入进入饱和区，导数接近 0，梯度消失，网络无法更新。
- **初始化过小**：每层输出方差不断缩小，激活值趋近于 0，信号在前向传播中逐渐”消亡”，网络同样无法训练。
- **全同初始化的对称性陷阱**：如果同一层所有神经元权重相同，正向传播时输出相同，反向传播时梯度相同，更新后仍然相同——这层退化成”一个神经元”，无论多宽都形同虚设。
- **Xavier/He 的共同目标**：让信号在前向传播中不放大、不缩小地流过每一层，保持激活值的方差在合理范围内。

参考答案与详细解析

(1) 六种初始化方法对比表：

初始化方法	规则	适用/注意事项
常数初始化	所有权重 $= c$	仅限偏置初始化为 0；权重禁用
秩初始化	低秩矩阵分解确定初值	特定结构网络，不常见
正态分布初始化	$\mathcal{N}(0, \sigma^2)$	σ 过大易梯度爆炸，过小易梯度消失
均匀分布初始化	$\mathcal{U}(-a, a)$	简单，未考虑网络深度/宽度
Xavier 初始化	$\sigma = \sqrt{2/(n_{\text{in}} + n_{\text{out}})}$	适合 Sigmoid/Tanh 激活函数
He 初始化	$\sigma = \sqrt{2/n_{\text{in}}}$	专为 ReLU 及其变体设计

(2) 常数初始化失效的原因（对称性问题）：

若同一层所有权重 $w_{ij} = c$，则每个神经元接收到完全相同的加权输入，输出完全相同（正向对称）；反向传播时各神经元梯度也完全相同（梯度对称），参数更新方向和幅度一致，更新后权重仍然相同。整层退化为等效的”一个神经元”，无法学习不同特征。

偏置可以初始化为 0：偏置不与同层神经元”对称连接”，不同神经元的偏置独立更新，初始全 0 不会导致对称性问题，且有助于在训练初期让激活函数在线性区域工作。

(3) Xavier 同时考虑输入和输出维度的原因：

在前向传播中，输入维度 n_{in} 决定了方差的放大倍数；在反向传播中，输出维度 n_{out} 决定了梯度方差的放大倍数。Xavier 取两者的调和平均 $(n_{\text{in}} + n_{\text{out}})/2$ 作为分母，使得前向和反向传播的方差都尽可能保持不变，是一种兼顾正向和反向信号传播的折中方案。

(4) He 初始化与 ReLU 的配合：

ReLU 对所有负输入输出 0，相当于在每次前向传播中”关掉”约一半的神经元，使有效传播的信号方差缩小约一半。若使用 Xavier 初始化（设计时假设激活函数不截断信号），每层输出方差会因 ReLU

的截断效应持续减半，导致深层信号趋近于 0。He 初始化将方差放大一倍（$2/n_{\text{in}}$），恰好补偿 ReLU 使方差减半的效果，使各层激活值的方差保持稳定。

初始化方法选择口诀

- **Sigmoid / Tanh ⇒ Xavier 初始化**（方差 $= 2/(n_{\text{in}} + n_{\text{out}})$）
- **ReLU / Leaky ReLU ⇒ He 初始化**（方差 $= 2/n_{\text{in}}$）
- **权重**：永远不要全常数初始化；**偏置**：通常初始化为 0

Q5-05 Batch Normalization 代码追踪与行为分析

题目描述：

文档第 5.4.1 节介绍了批量标准化（Batch Normalization，BN）。BN 是现代深度网络的”标配”组件，但其训练和推理行为的差异是一个常见的踩坑点。

以下是一段含 BN 层的网络代码：

```python
import torch
import torch.nn as nn

class BNNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1     = nn.Linear(4, 8)
        self.bn1     = nn.BatchNorm1d(8)
        self.relu    = nn.ReLU()
        self.fc2     = nn.Linear(8, 2)

    def forward(self, x):
        x = self.fc1(x)       # 步骤1
        x = self.bn1(x)       # 步骤2
        x = self.relu(x)      # 步骤3
        x = self.fc2(x)       # 步骤4
        return x

model = BNNet()
x_train = torch.randn(32, 4)  # batch_size=32, features=4

```

问题：

(1) **BN 的计算过程**：步骤 2 中，`bn1` 对输入 **z**（形状 `[32, 8]`）进行了哪些操作？写出完整的数学公式，并说明每一步的作用。

(2) **训练 vs 推理的差异**：`nn.BatchNorm1d` 在训练阶段和推理阶段的行为有何不同？推理时使用的均值和方差从哪里来？

(3) **BN 的位置**：BN 层放在线性层**之后**、激活函数**之前**是推荐做法，但也有观点认为放在激活函数之后效果更好。分别说明两种方式的理由，以及实践中的推荐选择。

(4) **BN 的可学习参数**：`bn1` 中有两个可学习参数 γ（缩放）和 β（平移）。解释为什么归一化之后还要进行缩放和平移？若没有这两个参数，BN 会丧失什么能力？

(5) 以下代码中，哪一行会导致 BN 行为异常？原因是什么？

```python
model.train()
for x_batch, y_batch in dataloader:
    pred = model(x_batch)
    loss = criterion(pred, y_batch)
    loss.backward()
    optimizer.step()

```

小白必读：BN 解决了什么问题？

- **内部协变量偏移 (Internal Covariate Shift)**：每一层的输入分布会随着上一层参数的更新而变化，导致每一层都要不断适应新的分布，训练极不稳定。BN 在每层输入处强制将激活值标准化为均值 0、方差 1（再通过 γ、β 调整），使分布保持相对稳定。
- **允许使用更大学习率**：没有 BN 时，过大的学习率容易让激活值进入饱和区，导致梯度消失；BN 将激活值控制在合理范围，即使学习率较大也不容易爆炸。
- **轻微的正则化效果**：BN 利用 mini-batch 的均值和方差来标准化，引入了一定的随机性（不同 batch 的统计量不同），类似于 Dropout 的噪声效果，有轻微防止过拟合的作用。

参考答案与详细解析

(1) BN 的完整计算过程：

设输入 **z** 形状为 `[32, 8]`（32 个样本，8 个特征），BN 对每个特征维度（每列）独立进行：

第一步：计算 mini-batch 均值

$$\mu_j = \frac{1}{N}\sum_{i=1}^{N} z_{ij}, \quad j = 1, \ldots, 8$$

第二步：计算 mini-batch 方差

$$\sigma_j^2 = \frac{1}{N}\sum_{i=1}^{N}(z_{ij} - \mu_j)^2$$

第三步：归一化

$$\hat{z}_{ij} = \frac{z_{ij} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

第四步：缩放与平移（γ_j、β_j 为可学习参数）

$$y_{ij} = \gamma_j \hat{z}_{ij} + \beta_j$$

(2) 训练 vs 推理的差异：

- **训练阶段**：使用当前 mini-batch 的均值 μ 和方差 σ^2 进行归一化，同时用指数移动平均维护全局的 `running_mean` 和 `running_var`；
- **推理阶段**：不再依赖当前 batch（推理时可能 batch_size=1），改用训练过程中积累的 `running_mean` 和 `running_var` 进行归一化，保证输出的确定性。

(3) BN 的位置：

- **线性层之后、激活函数之前**（原始论文推荐）：归一化的是线性变换的输出（未激活），数值分布最规则，归一化效果最稳定；
- **激活函数之后**：有研究表明在某些任务（如残差网络）中效果更好，但并非通用结论。

实践推荐：从"Linear → BN → ReLU" 开始，若效果不理想再尝试其他顺序。

(4) γ 和 β 的必要性：

归一化后所有层的激活值都被强制压缩到均值 0、方差 1 的范围，丧失了原本可能很有用的分布形态（比如某层本来激活值应该集中在正值区域）。γ 和 β 是可学习的，网络可以通过训练自动决定" 需要多大范围、需要偏移多少"，从而在标准化的稳定性和表达能力之间取得最优平衡。若去掉这两个参

数，网络的表达能力会受到严格限制，极端情况下甚至退化（当 $\gamma = 1$，$\beta = 0$ 恰好是纯归一化，是 BN 能学到的一种特殊情况）。

(5) 代码问题分析：

行 X 会导致 BN 行为异常。训练结束后模型仍处于 `train()` 模式，BN 层在推理时会继续用 `x_test` 这一个 batch 的统计量进行归一化，而非使用训练积累的 `running_mean`/`running_var`。若测试集只有少量样本（甚至单个样本），统计量极不稳定，输出严重失真。

修复方法：在推理前调用 `model.eval()`：

```
model.eval()
with torch.no_grad():
    pred_test = model(x_test)

```

Q5-06 正则化三剑客：权值衰减、Dropout 与 BatchNorm

题目描述：

文档第 5.4 节介绍了三种常见正则化手段。过拟合是深度学习实践中最常见的问题，三种方法从不同角度对其发起攻击。

场景：你训练了一个 5 层全连接网络用于图像分类，发现训练集准确率 98%，验证集准确率仅 72%，明显过拟合。

问题：

(1) **权值衰减（L2 正则化）**：在损失函数中加入参数的 L2 范数惩罚项：

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda \sum_{l} \|W^{(l)}\|_F^2$$

解释这个惩罚项如何在数学上抑制过拟合，以及 λ 过大或过小分别会带来什么问题。

(2) **Dropout**：训练时以概率 p 随机将神经元输出置零，推理时关闭 Dropout。阅读以下代码，指出其中的错误并改正：

```python
class MyNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1     = nn.Linear(128, 64)
        self.dropout = nn.Dropout(p=0.5)
        self.fc2     = nn.Linear(64, 10)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = MyNet()
# 训练循环
model.train()
for x_batch, y_batch in train_loader:
    pred = model(x_batch)
    loss = criterion(pred, y_batch)
    loss.backward()
    optimizer.step()

# 验证
for x_val, y_val in val_loader:
    pred_val = model(x_val)   # 错误：此处 Dropout 仍在生效
    val_loss = criterion(pred_val, y_val)

```

(3) **三种方法的作用机制对比**：填写下表，从”如何抑制过拟合”的角度分析三种方法：

方法	抑制过拟合的机制	额外副作用/优点

小白必读：过拟合是什么？为什么深度网络特别容易过拟合？

- **过拟合的本质**：模型参数太多、数据太少时，模型有”余力”去记住训练数据中的噪声（如某张猫图有一个特殊的划痕，模型记住”划痕 = 猫”），而不是学习真正的规律。测试集上没有这些噪声，泛化失败。
- **深度网络为何更容易过拟合**：参数量往往是数据量的数十乃至数百倍。一个 1000 万参数的网络，理论上可以”完美记忆”数万张图片，但这种记忆对新数据毫无用处。
- **正则化的共同目标：限制模型的”自由度”**，让它不得不学习泛化规律，而不是死记硬背训练集。

生活比喻：三种正则化像三种考试备考策略

- **权值衰减** = 限制复习时间（参数不能太大），逼你把时间用在最重要的知识点上，而不是背每道题的奇怪细节；
- **Dropout** = 复习时随机遮住一些笔记（随机关闭神经元），逼你从不同角度理解同一知识点，不依赖某一张特定的笔记页；
- **BatchNorm** = 定期对笔记进行整理归档（归一化激活值），让知识体系保持稳定，不让某一个知识点”权重”过大，顺带有轻微防死记的效果。

参考答案与详细解析

(1) 权值衰减的数学机制：

惩罚项 $\lambda\|W\|^2$ 对大权重施加额外惩罚，促使优化器倾向于选择更小的权重。更新时梯度变为：

$$\frac{\partial \mathcal{L}_{\text{total}}}{\partial W} = \frac{\partial \mathcal{L}_{\text{task}}}{\partial W} + 2\lambda W$$

等效于每次更新前将权重乘以 $(1 - 2\lambda\eta)$ 进行缩放（”衰减”）。权重越小，模型对输入的敏感性越低，对训练集噪声的过度拟合能力越弱。

- λ **过大**：权重被压缩过度，模型欠拟合，连训练集规律都学不到；
- λ **过小**：正则化效果微弱，过拟合依然存在。

常用范围：$\lambda \in [10^{-4}, 10^{-2}]$，具体由验证集表现决定。

(2) Dropout 代码错误与修正：

错误：验证循环中未切换到 `eval()` 模式，Dropout 仍以 $p = 0.5$ 随机关闭神经元，导致验证集的 Loss 和准确率每次都不一样，无法正确评估模型性能。

修正：

```python
# 验证前切换模式，并关闭梯度计算
model.eval()
with torch.no_grad():
    for x_val, y_val in val_loader:
        pred_val = model(x_val)   # Dropout 已关闭
        val_loss = criterion(pred_val, y_val)
# 验证结束后切回训练模式
model.train()

```

(3) 三种方法的作用机制对比表：

方法	抑制过拟合的机制	额外副作用/优点
权值衰减	惩罚过大权重，迫使模型保持简单	超参数 λ 需调节，可能欠拟合
Dropout	随机关闭神经元，迫使网络不依赖某些特定特征	相当于集成指数级个不同结构的子网络
BatchNorm	通过归一化引入 mini-batch 噪声，间接正则化	主要功能是加速训练/稳定梯度，正则化是副产品

(4) 综合解决方案：

三种方法**可以同时使用**，通常不存在根本冲突，但需注意：

- **BatchNorm + Dropout 的顺序**：通常先 BN 后 Dropout，且若网络已有 BN，Dropout 的 p 值可适当减小（如 0.3），因为 BN 本身已提供一定正则化；
- **推荐配置**（针对本题场景）：

```
# 每个隐藏层结构：Linear -> BN -> ReLU -> Dropout
nn.Linear(in, out),
nn.BatchNorm1d(out),
nn.ReLU(),
nn.Dropout(p=0.3),        # 适度 Dropout，不要 p=0.5 过强

```

- **权值衰减**：在优化器中加入 `weight_decay=1e-4`；
- **注意事项**：三种方法都会使训练 Loss 高于”没有正则化”时的值，这是正常的——目标是验证集 Loss，不是训练集 Loss。正则化强度过大会导致欠拟合，需通过验证集调参。

过拟合处置优先级建议

- **第一步**：增加数据量（最根本的解决方案）；
- **第二步**：加入 Dropout（$p = 0.3 \sim 0.5$）；
- **第三步**：加入权值衰减（$\lambda = 10^{-4} \sim 10^{-3}$）；
- **第四步**：加入 BatchNorm（同时加速训练）；
- **第五步**：考虑减小模型规模（减少层数/宽度）。

现实应用场景

这些”技巧”在工业界的实际地位：

本章所有内容在工业级深度学习中都是标配，而非可选项。以 GPT 系列大模型为例：它使用 Adam 优化器（带权值衰减的 AdamW 变体）、余弦退火学习率调度、He 初始化、LayerNorm（BatchNorm 的序列友好变体）。ResNet 之所以能训练到上百层，核心就是 BatchNorm 解决了深层网络的梯度问题，以及残差连接提供了额外的梯度通路。每一个本章的”技巧”，都是研究者在反复踩坑之后总结出来的，理解它们背后的原理，才能在遇到新问题时知道该用哪把钥匙。

2 本章小结：把”技巧”变成本能

学完这 6 道题，如果还觉得”内容繁杂、难以归纳”，试试把整章内容浓缩成这几句画面：

- **梯度消失/爆炸** = 深层网络的”信号失真”问题；换 ReLU、用合适初始化、加 BatchNorm，三管齐下。
- **优化器进化链**：SGD → Momentum（加惯性）→ AdaGrad（自适应步长）→ RMSProp（修复学习率归零）→ Adam（全都要）；学习率衰减是所有优化器的好搭档。
- **初始化的铁律**：权重不能全同；Sigmoid/Tanh 用 Xavier，ReLU 用 He；偏置初始化为 0。
- **BN 的核心**：训练用 batch 统计量，推理用 running 统计量，推理前必须 `model.eval()`！
- **正则化三剑客**：权值衰减限制权重大小，Dropout 阻断对特定神经元的依赖，BN 通过归一化噪声正则化——三者可以叠加使用。
- **过拟合的终极诊断**：训练集 Loss 低、验证集 Loss 高 ⇒ 过拟合，加正则化；两者都高 ⇒ 欠拟合，减正则化或增大模型。

给零基础读者的一句总比喻

如果把训练深度网络比作培养一名运动员：

- **梯度消失/爆炸** = 训练信号失真，教练（梯度）的指令传不到运动员耳里，或者喊得太大声把运动员吓到——换一套传令系统（ReLU + 合理初始化）；
- **优化器** = 训练计划：只跟着当天状态练（SGD），还是结合历史成绩制定长期方案（Adam）？
- **初始化** = 运动员的起跑姿势：一开始站对了，才能跑得快；
- **BatchNorm** = 每次训练前的热身和拉伸，让身体保持在最佳状态再开始发力；
- **正则化** = 防止过度专项训练：光练 100 米短跑（过拟合训练集），遇到障碍跑就废了；要练综合素质（泛化能力），才能在任何比赛中都有好成绩。